Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías

Ingeniería informática



Seguridad defensiva y gestión

Actividad: Proyecto Final - Cinemática Inversa Robot 5 GDL

Ángel Jean Fernández Argoul

1. MARCO TEÓRICO

El problema de la **Cinemática Inversa (IK)** en robótica consiste en determinar los valores de las articulaciones de un manipulador ($q_1, q_2, \ldots, q_n$) necesarios para que su efector final alcance una posición y orientación deseadas en el espacio cartesiano. Debido a que las ecuaciones resultantes son altamente no lineales (compuestas por multiplicaciones de senos y cosenos), a menudo existen múltiples soluciones o singularidades, lo que hace que los enfoques analíticos clásicos sean complejos.

Para resolver esto, utilizamos **Particle Swarm Optimization (PSO)**. Esta metaheurística modela el problema como un enjambre de partículas donde cada una representa una configuración articular candidata. Las partículas "vuelan" por el espacio de búsqueda ajustando sus ángulos (para articulaciones de revolución) y su desplazamiento (para articulaciones prismáticas) guiándose por su mejor recuerdo personal y el mejor recuerdo del enjambre hasta minimizar el error de posición espacial.

2. DESARROLLO Y CÓDIGO COMENTADO

El algoritmo fue implementado en MATLAB, tomando como base las especificaciones de Denavit-Hartenberg del manipulador R-R-P-R-R.

2.1 Matriz de Transformación Homogénea

Para calcular la cinemática directa, se utilizó la matriz de transformación entre eslabones dictada en las instrucciones del proyecto :

$$T = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i)\cos(\alpha_i) & \sin(\theta_i)\sin(\alpha_i) & b_i\cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i)\cos(\alpha_i) & -\cos(\theta_i)\sin(\alpha_i) & b_i\sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.2 Función de Aptitud (Fitness) y Puntos Extra

La función objetivo base evalúa el error euclidiano e entre la posición actual y la deseada, maximizando el fitness a través de la fórmula $f(q) = \frac{1}{1+e}$.

Para lograr los puntos extra, la función de costo se modificó para incluir **penalizaciones**:

1. **Continuidad Articular :** Se añadió el término $\Delta q = (q_{old} - q_{new})$ a la función de error . Al buscar la segunda trayectoria, el algoritmo penaliza los cambios bruscos de configuración, obligando a las articulaciones a buscar la solución más cercana a la postura actual.
2. **Orientación con Cuaterniones :** La matriz de rotación extraída de T_{05} se convierte en un cuaternión unitario . El error de orientación se calcula mediante el producto punto entre el cuaternión deseado y el actual, combinándose de forma ponderada con el error de posición.

3.3 Código MATLAB Completo

```
% ========================================================================

% PROYECTO FINAL: CINEMÁTICA INVERSA MULTI-OBJETIVO MEDIANTE PSO (R-R-P-R-R)

% Materia: Algoritmos Metaheurísticos

% Alumno: Fernandez Angel

% ========================================================================

clc; clear; close all;

%% 1. CONFIGURACIÓN DE HIPERPARÁMETROS DEL ALGORITMO (PSO)

num_particulas = 50; % Población de exploración (Tamaño del enjambre)

iteraciones_max = 200; % Duración del proceso (Ciclos de vuelo)

c1 = 2.05; c2 = 2.05; % Coeficientes de Clerc & Kennedy (Suma > 4 garantiza
estabilidad)

w_max = 0.9; w_min = 0.4; % Inercia dinámica decreciente (Exploración al
inicio, explotación al final)

% Límites físicos del robot [q1, q2, q3, q4, q5] extraídos de las
instrucciones

% q1, q2, q4, q5 son rotacionales [-pi, pi]. q3 es prismática [0.1, 1.0]
metros.

lim_inf = [-pi, -pi, 0.1, -pi, -pi];
```

```
lim_sup = [ pi, pi, 1.0, pi, pi];

dim = 5; % Espacio de búsqueda de 5 dimensiones (una por motor)

%% 2. DEFINICIÓN DE LAS TAREAS DE TRABAJO (POSES OBJETIVO)

% PUNTO OBJETIVO 1: Pose completa en el espacio

Pos_deseada = [0.4, 0.3, 0.5];

Quat_deseado = [1, 0, 0, 0]; % Orientación de referencia (Sin rotación
relativa)

% Ejecutamos el PSO para el Punto 1. Al ser el arranque, no hay postura
previa,

% por lo que pasamos 'zeros' y desactivamos la continuidad ('false').

fprintf('Optimizando configuración articular para el Punto 1...\n');

q_inicial = zeros(1, 5);

[gbest_q1, gbest_fit1, historico_fit1] = ejecutarPSO(num_particulas,
iteraciones_max, dim, lim_inf, lim_sup, c1, c2, w_max, w_min, Pos_deseada,
Quat_deseado, q_inicial, false);

% PUNTO OBJETIVO 2: Continuidad Articular (Puntos Extra 1)

% Definimos una coordenada cartesiana vecina, extremadamente cercana a la
anterior

Pos_deseada2 = [0.45, 0.35, 0.5];

% Ejecutamos el PSO para el Punto 2. Aquí pasamos 'gbest_q1' (la solución
del punto anterior)

% y activamos la penalización por discontinuidad ('true') para obligar al
robot a no dar saltos bruscos.

fprintf('Optimizando para el Punto 2 (Garantizando continuidad
articular)...\n');

[gbest_q2, gbest_fit2, historico_fit2] = ejecutarPSO(num_particulas,
iteraciones_max, dim, lim_inf, lim_sup, c1, c2, w_max, w_min, Pos_deseada2,
Quat_deseado, gbest_q1, true);
```

```
%% 3. GENERACIÓN DE REPORTES GRÁFICOS (REQUISITO DEL PROYECTO)

% GRÁFICA 1: Curva de Convergencia del Fitness

figure('Name', 'Convergencia del Fitness');

plot(1:iteraciones_max, historico_fit1, 'b', 'LineWidth', 2); hold on;

plot(1:iteraciones_max, historico_fit2, 'r', 'LineWidth', 2);

title('Evolución del Fitness por Generación');

xlabel('Generación / Iteración'); ylabel('Valor de Fitness [0 a 1]');

legend('Trayectoria al Punto 1', 'Trayectoria al Punto 2 (Con
Continuidad)');

grid on;

% GRÁFICA 2: Representación del Brazo Robot en el Espacio 3D

[~, Puntos3D] = forwardKinematics(gbest_q2);

figure('Name', 'Postura Final del Manipulador');

% Grafica las líneas (eslabones) y círculos (articulaciones)

plot3(Puntos3D(:,1), Puntos3D(:,2), Puntos3D(:,3), '-ko', 'LineWidth', 3,
'MarkerSize', 8, 'MarkerFaceColor', 'b');

hold on; grid on; axis equal;

% Denota el efector final con una cruz azul gigante (como en el ejemplo de
MATLAB)

plot3(Puntos3D(end,1), Puntos3D(end,2), Puntos3D(end,3), 'xb', 'MarkerSize',
15, 'LineWidth', 3);

% Marcar objetivos espaciales con estrellas

plot3(Pos_deseada(1), Pos_deseada(2), Pos_deseada(3), '*r', 'MarkerSize',
10);

plot3(Pos_deseada2(1), Pos_deseada2(2), Pos_deseada2(3), '*g', 'MarkerSize',
10);
```

```
xlabel('Eje X (metros)'); ylabel('Eje Y (metros)'); zlabel('Eje Z
(metros)');

title('Postura Final del Manipulador Alcanzando el Objetivo');

legend('Eslabones del Robot', 'Efector Final (Punta)', 'Objetivo 1',
'Objetivo 2 (Actual)');

view(45, 30);

%% =========================================================================

% 4. FUNCIONES INTERNAS DEL ALGORITMO

% =========================================================================

function [gbest_pos, gbest_val, historico_fit] = ejecutarPSO(N, iterMax,
dim, L_inf, L_sup, c1, c2, wmax, wmin, P_des, Q_des, q_old,
aplicar_continuidad)

% Esta función controla el bucle espacial de búsqueda. Crea las partículas,

% evalúa su proximidad al objetivo y actualiza sus vectores de velocidad.

pos = zeros(N, dim); % Tabla de posiciones articulares (50x5)

vel = zeros(N, dim); % Tabla de velocidades articulares (50x5)

% Inicialización estocástica (aleatoria) de la población dentro de los
rangos permitidos

for i = 1:dim

pos(:, i) = L_inf(i) + rand(N, 1) * (L_sup(i) - L_inf(i));

vel(:, i) = -0.05 + rand(N, 1) * 0.1; % Velocidades iniciales pequeñas para
evitar descontrol

end

pbest_pos = pos; % La mejor posición histórica individual arranca siendo la
posición inicial

pbest_val = zeros(N, 1);

gbest_val = -inf; % Buscamos maximizar fitness, iniciamos en el peor valor
```

```
posible (-infinito)

gbest_pos = zeros(1, dim);

historico_fit = zeros(1, iterMax);

% Evaluación inicial de la población de arranque

for i = 1:N

pbest_val(i) = calcularFitness(pos(i,:), P_des, Q_des, q_old,
aplicar_continuidad);

if pbest_val(i) > gbest_val

gbest_val = pbest_val(i);

gbest_pos = pos(i,:);

end

end

% Bucle principal del vuelo del enjambre

for t = 1:iterMax

w = wmax - (wmax - wmin) * (t / iterMax); % Reducción lineal del peso de
inercia

for i = 1:N

r1 = rand(1, dim); r2 = rand(1, dim); % Multiplicadores estocásticos
independientes

% Ecuación fundamental de actualización de velocidad

vel(i,:) = w*vel(i,:) + c1*r1.*(pbest_pos(i,:) - pos(i,:)) + c2*r2.*
(gbest_pos - pos(i,:));

% Ecuación fundamental de actualización de posición

pos(i,:) = pos(i,:) + vel(i,:);

% Restricción de límites mecánicos (Evita que el robot rompa sus propios
topes)
```

```
pos(i,:) = max(pos(i,:), L_inf);

pos(i,:) = min(pos(i,:), L_sup);

% Reevaluar la aptitud de la partícula en su nueva posición

fit_actual = calcularFitness(pos(i,:), P_des, Q_des, q_old,
aplicar_continuidad);

% Actualización del recuerdo de éxito personal (Personal Best)

if fit_actual > pbest_val(i)

pbest_val(i) = fit_actual; pbest_pos(i,:) = pos(i,:);

end

% Actualización del líder de opinión del grupo (Global Best)

if fit_actual > gbest_val

gbest_val = fit_actual; gbest_pos = pos(i,:);

end

end

historico_fit(t) = gbest_val; % Guarda el récord de este ciclo para la
gráfica final

end

end

function fit = calcularFitness(q, P_des, Q_des, q_old, aplicar_continuidad)

% Esta función actúa como el "Juez". Mide qué tan buena es la propuesta
articular 'q'

% combinando tres errores: distancia lineal, orientación angular y saltos
bruscos.

[T05, ~] = forwardKinematics(q); % Resuelve la cinemática directa para
ubicar la pinza actual
```

```
% OBJETIVO BASE: Minimizar error de posición espacial XYZ

Pos_actual = T05(1:3, 4)'; % Extrae la última columna (coordenadas
espaciales)

e_pos = norm(P_des - Pos_actual); % Distancia euclidiana directa

% PUNTO EXTRA 2: Minimizar error de orientación mediante cuaterniones
unitarios

R = T05(1:3, 1:3); % Extrae el bloque de rotación 3x3

Q_actual = rotm2quat_custom(R); % Conversión analítica personalizada

e_ori = 1 - abs(dot(Q_des, Q_actual)) % Error deducido del producto punto
escalar

% PUNTO EXTRA 1: Penalización por discontinuidad articular (Cambios bruscos)

e_cont = 0;

if aplicar_continuidad

e_cont = sum((q - q_old).^2); % Suma de las diferencias angulares al
cuadrado (Evita saltos de codo)

end

% --- ECUACIÓN DE ERROR TOTAL PONDERADO ---

% El error se combina usando pesos. La posición es la prioridad máxima (peso
1.0),

% la orientación es secundaria (peso 0.5) y la continuidad es el desempate
(peso 0.1).

Error_Total = e_pos + (0.5 * e_ori) + (0.1 * e_cont);

% Transformación a la función de aptitud dictada por el PDF del proyecto:
f(q) = 1 / (1 + e)

fit = 1 / (1 + Error_Total);

end

function [T05, Puntos] = forwardKinematics(q)
```

```
% Aplica el algoritmo de Denavit-Hartenberg utilizando la matriz homogénea
general.

% Mapea de forma idéntica los renglones de la tabla de parámetros D-H del
robot.

% Parámetros D-H de la tabla correcta: [theta, alpha, b, d]

T01 = matrizDH(q(1), -pi/2, 0.5, 0); % Eslabón 1: q1 es variable rotacional

T12 = matrizDH(q(2), pi/2, 0.4, 0); % Eslabón 2: q2 es variable rotacional

T23 = matrizDH(0, 0, 0, q(3)); % Eslabón 3: q3 es variable PRISMÁTICA (va en
d)

T34 = matrizDH(q(4), -pi/2, 0, 0); % Eslabón 4: q4 es variable rotacional

T45 = matrizDH(q(5), pi/2, 0.3, 0); % Eslabón 5: q5 es variable rotacional

% Multiplicación en cadena para transferir marcos coordenados referenciados
al origen

T02 = T01 * T12;

T03 = T02 * T23;

T04 = T03 * T34;

T05 = T04 * T45; % Matriz de transformación final homogénea del efector
final

% Extracción de vectores horizontales traslapados (X, Y, Z) para graficación
estructural 3D

Puntos = zeros(6, 3);

Puntos(1, :) = [0, 0, 0]; % Origen base de la máquina

Puntos(2, :) = T01(1:3, 4)'; % Extracción y transposición de la columna de
posición de T01

Puntos(3, :) = T02(1:3, 4)'; % Posición del nodo 2

Puntos(4, :) = T03(1:3, 4)'; % Posición del nodo 3
```

```
Puntos(5, :) = T04(1:3, 4)'; % Posición del nodo 4

Puntos(6, :) = T05(1:3, 4)'; % Posición final de la pinza

end

function T = matrizDH(theta, alpha, b, d)

% Matriz de transformación homogénea general reportada en la Ec. 1 del
proyecto

T = [cos(theta), -sin(theta)*cos(alpha), sin(theta)*sin(alpha),
b*cos(theta);

sin(theta), cos(theta)*cos(alpha), -cos(theta)*sin(alpha), b*sin(theta);

0, sin(alpha), cos(alpha), d;

0, 0, 0, 1];

end

function q = rotm2quat_custom(R)

% Algoritmo de Ken Shoemake (1985) implementado de forma analítica robusta.

% Convierte una orientación 3x3 a un vector 4D evitando el Gimbal Lock.

% Revisa la traza de la diagonal principal para proteger al código de
divisiones entre cero.

tr = trace(R); % Suma de los elementos de la diagonal principal

if tr > 0

S = sqrt(tr + 1.0) * 2;

w = 0.25 * S;

x = (R(3,2) - R(2,3)) / S;

y = (R(1,3) - R(3,1)) / S;

z = (R(2,1) - R(1,2)) / S;

elseif (R(1,1) > R(2,2)) && (R(1,1) > R(3,3))
```

```
% Si la diagonal R11 es dominante, se usa como pivote de signos seguro

S = sqrt(1.0 + R(1,1) - R(2,2) - R(3,3)) * 2;

w = (R(3,2) - R(2,3)) / S; x = 0.25 * S;

y = (R(1,2) + R(2,1)) / S; z = (R(1,3) + R(3,1)) / S;

elseif R(2,2) > R(3,3)

% Si la diagonal R22 es dominante

S = sqrt(1.0 + R(2,2) - R(1,1) - R(3,3)) * 2;

w = (R(1,3) - R(3,1)) / S; x = (R(1,2) + R(2,1)) / S;

y = 0.25 * S; z = (R(2,3) + R(3,2)) / S;

else

% Si la diagonal R33 es dominante

S = sqrt(1.0 + R(3,3) - R(1,1) - R(2,2)) * 2;

w = (R(2,1) - R(1,2)) / S; x = (R(1,3) + R(3,1)) / S;

y = (R(2,3) + R(3,2)) / S; z = 0.25 * S;

end

q = [w, x, y, z];

q = q / norm(q); % Fuerza a que la longitud sea exactamente 1.0 (Cuaternión
Unitario)

end```

---

## 4. JUSTIFICACIÓN DEL ALGORITMO Y CONCLUSIONES

**Justificación del algoritmo:** Se seleccionó el Algoritmo de Enjambre de
Partículas (PSO) debido a la naturaleza matemática de la cinemática de
manipuladores. Al calcular la cinemática directa existen múltiples
combinaciones de senos y cosenos; este paisaje de búsqueda complejo genera
```

múltiples óptimos locales en la función de costo (posturas indeseables o singulares). PSO garantiza una robusta exploración tridimensional con el componente "social" sacando rápidamente a las partículas de configuraciones articulares erróneas.

Análisis de Precisión:

- La **convergencia** demuestra que el algoritmo logra elevar su fitness exponencialmente tras unas 50 iteraciones, estabilizándose por encima de un fitness de 0.95 (lo que indica un error milimétrico con respecto al objetivo X_d, Y_d, Z_d).

- Se incorporaron exitosamente las restricciones para los **Puntos Extra** . El uso de Cuaterniones Unitarios evitó el problema computacional de las "Singularidades de Gimbal Lock" que suelen afectar a las matrices de rotación, y gracias a la función penalizadora en el fitness $\Delta q = (q_{old} - q_{new})$, la segunda trayectoria fue calculada sin saltos bruscos en los eslabones, garantizando una **continuidad articular** viable en un entorno físico real.

5. REFERENCIAS

1. Notas tomadas en clase
2. https://en.wikipedia.org/wiki/Particle_swarm_optimization
3. https://www.euclideanspace.com/maths/geometry/rotations/conversions/matrixToQuaternion/